

GRUPO I - 1) `static Pair<Skill,String> temSkillAleatoria(List<Pessoa> listaP)`

1. Escolhe uma `Skill` aleatória usando `Skill.values()` e `Random.nextInt(n)`.
2. Percorre `listaP` e junta (com `StringBuilder`) os **nomes** das pessoas que têm essa skill.
3. Separa os nomes por `;`
4. Devolve `new Pair<>(skillEscolhida, stringComNomes)`.

```
static Pair<Skill, String> temSkillAleatoria(List<Pessoa> listaP) {
    Skill[] skills = Skill.values();
    Random r = new Random();

    Skill escolhida = skills[r.nextInt(skills.length)];

    StringBuilder sb = new StringBuilder();

    for (Pessoa p : listaP) {
        if (p.hasSkill(escolhida)) {
            sb.append(p.name());
            sb.append(";");
        }
    }

    return new Pair<>(escolhida, sb.toString());
}
```

2) `AbstractGame` e `FirstGuessGame`

a) Instruções (game1)

i) Que atributos tem o objeto `game1`?

Tem os da superclasse + os da subclasse:

De `AbstractGame`: `minG`, `maxG`, `current`, `treasure`, `players`, `generator`

De `FirstGuessGame`: `finish`, `win`

ii) Valores antes da instrução 4 (tesouro = 5)

Depois das linhas 1–3:

- `players = ["Maria", "Pedro"]`
- `current = -1`
- `finish = false`
- `win = null`

iii) Valores após a instrução 4 (supondo `nextMove()` devolve 5)

`System.out.println(game1.nextMove());`

- `current` passa de `-1` para `0`
- jogador atual = `"Maria"`
- 2. Como `treasureFound(5)` é `true`:
- `win = "Maria"`
- `finish = true`

Imprime: 5

b) Instruções (game2)

i) Que atributos tem o objeto `game2`?

Os mesmos do `game1` (super + sub), mesmo estando referenciado por `AbstractGame`.

ii) Valores antes da instrução 2 (tesouro = 5)

- `players = []`
- `current = -1`
- `finish = false`
- `win = null`

iii) Output final (se `nextMove()` der 8, depois 4, depois 5)

(`nextMove()` devolve 8, depois 4, depois 5)

8

4

5

Hugo

GRUPO II - 1) Classe abstrata **Cooperativa** (implementação parcial)

- construtor **Cooperativa(String nome)** com lista vazia de associados
- **aceitaTarefa(...)** como **template**
- **recebeDonativo(...)** (crédito igual para todos)
- mais os métodos do diagrama como **nome()**, **associados()**, **novoAssociado(...)**, **toString()**

```
public abstract class Cooperativa implements ICooperativa {

    private String nome;
    private List<IAssociado> associados;

    public Cooperativa(String nome) {
        this.nome = nome;
        this.associados = new ArrayList<>();
    }

    public String nome() {
        return nome;
    }

    @Override
    public Iterable<IAssociado> associados() {
        return associados;
    }

    @Override
    public void novoAssociado(IAssociado a) {
        associados.add(a);
    }

    @Override
    public Iterable<String> aceitaTarefa(double custo, Skill[] skills) {
        // as subclasses definem como dividir custos e como dividir tarefa
        Iterable<Pair<String, Skill>> t = divideTarefa(skills);
        Iterable<Pair<String, Double>> c = divideCustos(custo);

        // devolver nomes de quem participou (união)
        List<String> nomes = new ArrayList<>();

        for (Pair<String, Skill> p : t) {
            if (!nomes.contains(p.first())) {
                nomes.add(p.first());
            }
        }

        for (Pair<String, Double> p : c) {
            if (!nomes.contains(p.first())) {
                nomes.add(p.first());
            }
        }

        return nomes;
    }

    @Override
    public void recebeDonativo(double quantia, Pair<Integer, Integer> anoMes) {
        int n = associados.size();
        if (n == 0) return;

        double parte = quantia / n;
        for (IAssociado a : associados)
            a.addCredit(parte, anoMes);
    }

    @Override
    public String toString() {
        return "Cooperativa: " + nome + " (associados=" + associados.size() + ")";
    }
}
```

2) Classe concreta CoopPP

- atributos: `taxaRet` e `fundosProprios`
- construtor `CoopPP(String nome, double fundoProp, int taxa)`
- implementar:
 - `divideCustos(double quantia)`
 - `divideTarefa(Skill[] skills)`
- redefinir `recebeDonativo(...)` e `toString()`
- ter `taxaRetencao()` e `fundosProprios()`

```
public class CoopPP extends Cooperativa {

    private int taxaRet;           // 0..100
    private double fundosProprios;

    public CoopPP(String nome, double fundoProp, int taxa) {
        super(nome);
        this.fundosProprios = fundoProp;
        this.taxaRet = taxa;
    }

    public int taxaRetencao() {
        return taxaRet;
    }

    public double fundosProprios() {
        return fundosProprios;
    }

    @Override
    public Iterable<Pair<String, Double>> divideCustos(double quantia) {
        List<Pair<String, Double>> res = new ArrayList<>();

        // ano corrente
        int anoAtual = Year.now().getValue();
        Pair<Integer, Integer> anoMes = new Pair<>(anoAtual, 1);

        // parte retirada dos fundos próprios
        double ret = (taxaRetencao() / 100.0) * quantia;
        double resto = quantia - ret;

        // associados que vão pagar o resto:
        // último débito em ano anterior ao ano corrente

        List<IAssociado> pagadores = new ArrayList<>();
        for (IAssociado a : associados()) {
            Pair<Integer, Integer> last = a.lastDebitYearMonth();
            if (last != null && last.first() < anoAtual) {
                pagadores.add(a);
            }
        }

        if (pagadores.isEmpty()) {
            // ninguém cumpre condição => tudo sai dos fundos próprios
            fundosProprios -= quantia;
            return res;
        }

        // fundos próprios pagam "ret"
        fundosProprios -= ret;

        double parte = resto / pagadores.size();
        for (IAssociado a : pagadores) {
            a.addDebit(parte, anoMes);
            res.add(new Pair<>(a.name(), parte));
        }
        return res;
    }
}
```

```

@Override
public Iterable<Pair<String, Skill>> divideTarefa(Skill[] skills) {
    List<Pair<String, Skill>> res = new ArrayList<>();
    for (IAssociado a : associados()) {
        for (Skill s : skills) {
            if (a.hasSkill(s)) {
                res.add(new Pair<>(a.name(), s));
            }
        }
    }
    return res;
}

@Override
public void recebeDonativo(double quantia, Pair<Integer, Integer> anoMes) {
    double paraFundos = (taxaRet / 100.0) * quantia;
    fundosProprios += paraFundos;

    double resto = quantia - paraFundos;
    super.recebeDonativo(resto, anoMes);
}

@Override
public String toString() {
    return super.toString() +
        " taxa=" + taxaRet +
        " fundos=" + fundosProprios;
}
}

```

3) equals em CoopPP: que outro método redefinir?

```

@Override
public boolean equals(Object o) {
    if (!super.equals(o)) return false;
    CoopPP other = (CoopPP) o;
    return taxaRet == other.taxaRet;
}

```

-Sempre que se redefine o método **equals**, deve também ser redefinido o método **hashCode**, de forma consistente.

GRUPO III - 1) Instruções incorretas

- **Linha 5:** `Cooperativa coop1 = new Cooperativa("Coop de Prata");`

Errado porque **Cooperativa** é abstrata (no enunciado do Grupo II diz “classe abstrata Cooperativa”).

Não se pode instanciar uma classe abstrata.

Correção: `Cooperativa coop1 = new CoopPP("Coop de Prata", 0.0, 0);`

- **Linha 13:** `List<ICooperativa> listaUm = new ArrayList<CoopPP>();`

Errado por causa de genéricos (invariância).

Mesmo que **CoopPP** implemente **ICooperativa**, `ArrayList<CoopPP>` NÃO é subtipo de `List<ICooperativa>`.

Correção: `List<ICooperativa> listaUm = new ArrayList<>();`

- **Linhas 22–24 (vão dar erro com a assinatura atual de imprimeResultados)**

`imprimeResultados(listaUm);`

`imprimeResultados(listaDois);`

`imprimeResultados(listaTres);`

`private static void imprimeResultados (Collection<ICooperativa> col)`

- `listaUm` é `List<ICooperativa>` → compatível com `Collection<ICooperativa>`
- `listaDois` é `ArrayList<Cooperativa>` → **NÃO** é `Collection<ICooperativa>`
- `listaTres` é `List<Cooperativa>` → também não é `Collection<ICooperativa>`

Genéricos são invariantes: `Collection<Cooperativa>` não “encaixa” em `Collection<ICooperativa>`.
Correção: `private static void imprimeResultados(Collection<? extends ICooperativa> col)`

2) Só linhas 22–24 e da 37 até ao fim

2.1 Há erros nessas partes?

Erro: `private static void imprimeResultados (Collection<ICooperativa> col)`

Faz com que `listaDois` e `listaTres` não sejam aceites.

Alteração: `private static void imprimeResultados(Collection<? extends ICooperativa> col)`

2.2 Depois de corrigir, o que é escrito no ecrã?

O método imprime:

```
for(ICooperativa coop : col) {  
    System.out.println(coop.toString());  
}
```

Então ele imprime **uma linha por cada elemento** da coleção.